

94 - Claims based authorization

Искаме даден потребител да има право на определено действие, само ако изрично е указано, че притежава това право.

Claims Based Authorization



Claims are policy based. We create a policy and include one or more claims in that policy. We then need to register the policy. Creating and registering a claims policy is typically done in one step in [ConfigureServices\(\)](#) method of the [Startup](#) class.

Implementing Claims Based Authorization

Step 1 : Create Claims Policy

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));
});
```

Step 2 : Use the Policy for Authorization Checks

```
[HttpPost]
[Authorize(Policy = "DeleteRolePolicy")]
public async Task<IActionResult> DeleteRole(string id)
{
    // Delete Role
}
```

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));
});
```

- The [options](#) parameter type is [AuthorizationOptions](#)
- Use [AddPolicy\(\)](#) method to create the policy
- The first parameter is the [name](#) of the policy and the second parameter is the policy itself
- To satisfy this policy requirements, the logged-in user must have [Delete Role](#) claim

Using Claims Policy for Authorization Checks

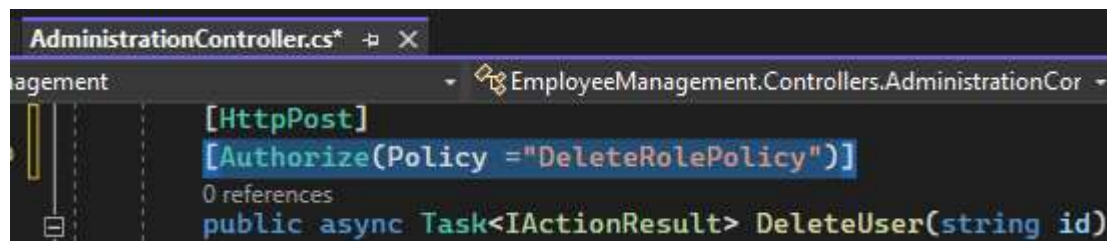
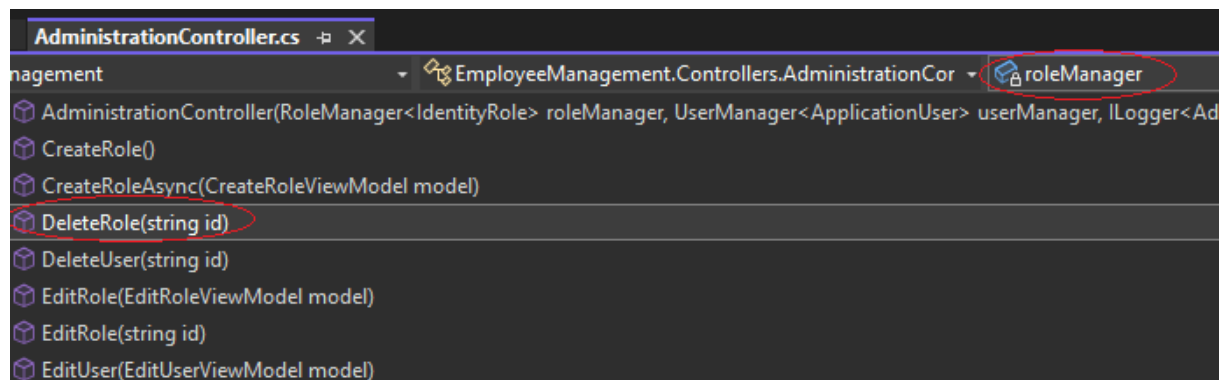
The policy can then be used on a controller or a controller action.

`[Authorize(Roles = " ")]`, аналогично

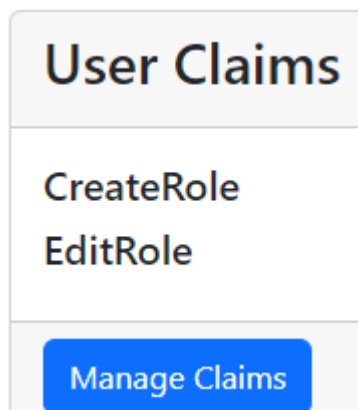
`[Authorize(Policy = " ")]`

`[Authorize(Policy = "DeleteRolePolicy")]`

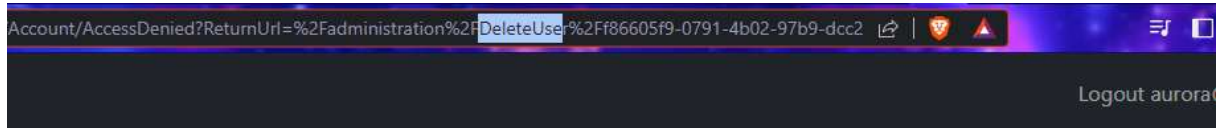
Интересното тук е падащото меню за бързо търсене на action в контролера:



Така, по подразбиране нямаме DeleteRole.



[Jason@abv.bg](#) = TestRole1, ако админ Amidala се опита да изтрие Jason, тя не е авторизирана.



Access Denied

You do not have permission to view this resource



User Claims

None at the moment

Manage Claims

Manage User Claims

- ☒ CreateRole
- ☒ EditRole
- ☒ DeleteRole

Update Cancel

TestRole1

Edit

Are you sure you want to delete?

Yes

No

Всеки подобен ъпдейт влиза в сила след отписване и повторно вписване на потребител.

Целият `AdministrationController` изисква `Admin` role, а отделно вътре в него `Delete` action изисква `DeleteRolePolicy`, което означава, че за да бъде изтрит потребител, той трябва да бъде едновременно `Admin` и да има изрично право да трие. Ако искаме да дадем допълнително изискване за политика, просто присъединяваме `.RequireClaim("Create Role")`; което означава, че само потребител, който има и двете права `Delete` + `Create` ще бъде допуснат до `DeleteRole`.

Adding Multiple Claims to Policy

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role")
            .RequireClaim("Create Role")
    );
});
```